Thadomal Shahani Engineering College

Departement of Artificial Intelligence and Data Science

Name: Krish Makhija	Roll no: 57	Branch: AI&DS
Subject : ASD&D	Batch: 13	Date: 10-10-2026

EXPERIMENT 9: To provision cloud infrastructure using Terraform.

Aim : To provision cloud infrastructure using Terraform.

LO Mapped : LO4

LO : Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

- IAM User / Account ID: Experiment7 / 702561770822
- Target AWS Region: ap-southeast-2 (Asia Pacific - Sydney)
- Deployment Status: Successfully Provisioned, Verified, and Deprovisioned

Aim / Objective:

To design, write, validate, and execute Infrastructure as Code (IaC) configurations using HashiCorp Terraform to dynamically provision, configure, verify, and destroy an Amazon Elastic Compute Cloud (EC2) virtual machine and associated resources on Amazon Web Services (AWS).

Theoretical Background:

What is Infrastructure as Code (IaC)?

Infrastructure as Code (IaC) is an IT infrastructure provisioning process where developers and DevOps engineers automatically manage and provision cloud resources (networks, virtual machines, load balancers, and storage) through human-readable definition files rather than manual user interface configurations. IaC enforces version control, repeatability, scalability, and consistency across environments.

Key Terraform Concepts & Lifecycle Commands

HashiCorp Terraform is an open-source IaC tool that uses HashiCorp Configuration Language (HCL) to define cloud infrastructure declarative state files.

- Declarative Configuration (.tf): Files where target cloud resources are defined. All

.tf files in a single execution directory are parsed together as a unified configuration.

- terraform init: Initializes the working directory, downloads required provider plugins

(e.g., hashicorp/aws v6.67.0), and creates dependency lock files (.terraform.lock.hcl).

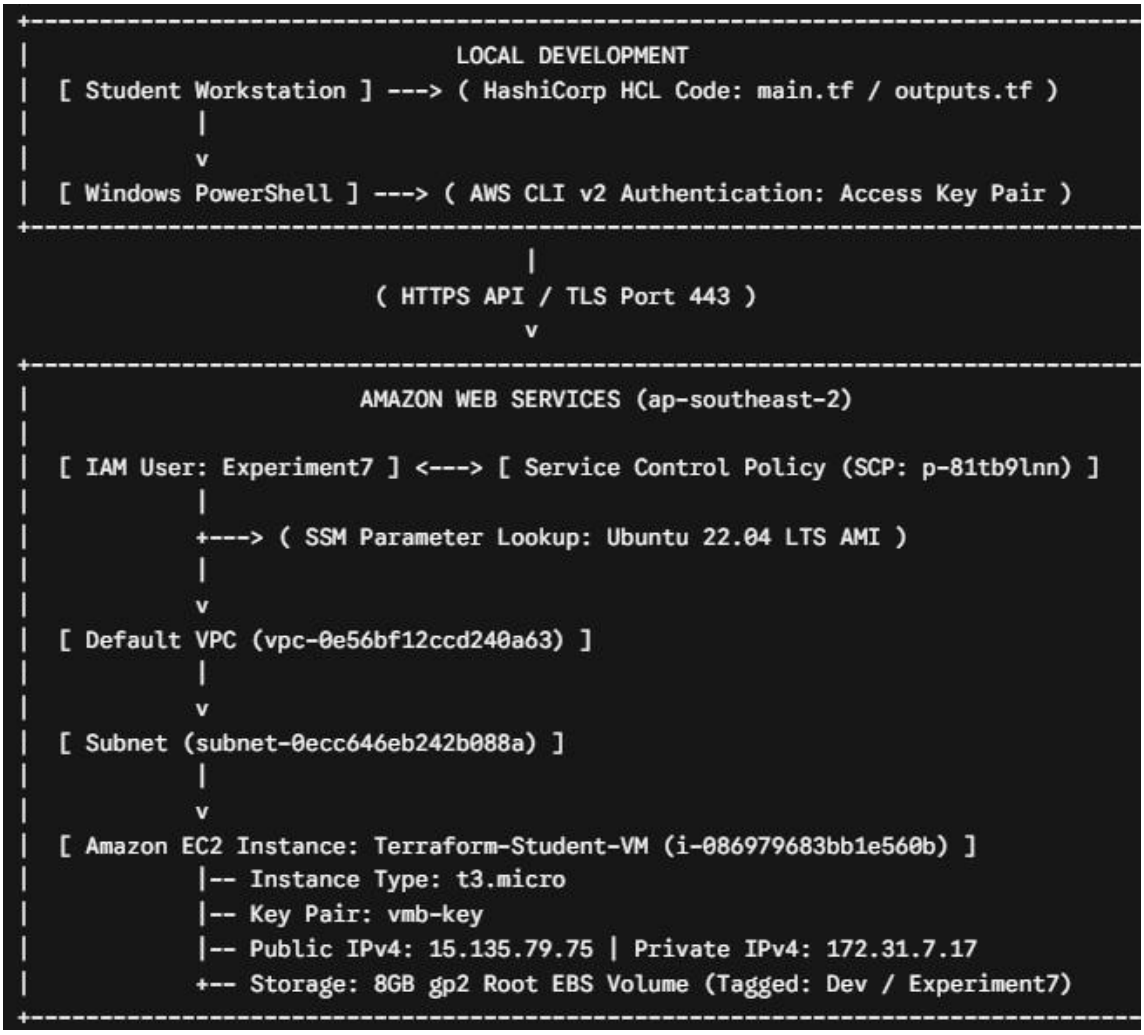
- terraform validate: Checks the syntax, structure, and internal consistency of HCL configuration files.
- terraform plan: Compares the local declarative configuration against the active cloud infrastructure state and constructs an execution plan without applying changes.
- terraform apply: Executes the proposed plan against AWS APIs to provision or modify cloud resources, saving state metadata into terraform.tfstate.
- terraform output: Extracts and displays defined output variables from the state file.
- terraform destroy: Deprovisions and cleans up all managed resources from AWS to prevent unnecessary billing.

Interfacing AWS Concepts & Security

1. AWS Identity and Access Management (IAM): Grants precise execution policies (AmazonEC2FullAccess, AmazonVPCFullAccess, AmazonSSMReadOnlyAccess) and security credentials (Access Key ID & Secret Access Key) for programmatic CLI access.
2. AWS Systems Manager (SSM) Parameter Store: Dynamically retrieves system parameters such as up-to-date, region-specific Canonical Ubuntu AMI IDs (/aws/service/canonical/ubuntu/server/22.04/stable/current/amd64/hvm/ebsgp2/ami id).

3. AWS Service Control Policies (SCPs): Guardrails enforced at the AWS Organization level (p-81tb9lnn) that mandate resource compliance, such as explicit resource/volume tags (Environment = "Dev", Project = "Experiment7"), allowed instance types (t3.micro), and attached SSH Key Pairs (vmb-key).

Architecture Diagram & Workflow



Declarative Configuration Code

The configuration code was organized inside C:\Terraform.

1. main.tf (Infrastructure Definition)

Terraform terraform {

required_providers { aws = { source =

"hashicorp/aws" version =

```
"~> 6.0"
    }
  }
}

provider "aws" {   region
= "ap-southeast-2"
}

# Dynamic SSM Parameter Lookup for Ubuntu 22.04 LTS AMI data "aws_ssm_parameter"
"ubuntu_ami" {   name        =
 "/aws/service/canonical/ubuntu/server/22.04/stable/current/amd64/hvm/ebsgp2/ami-id"
}

# Default VPC and Subnet Fetching
data "aws_vpc" "default" {   default
= true
}

data "aws_subnets" "default" {
filter {     name   = "vpc-id"
values = [data.aws_vpc.default.id]
  }
}
```

```
# EC2 Instance Resource Definition resource "aws_instance" "student_vm" {
  ami                    = data.aws_ssm_parameter.ubuntu_ami.value
  instance_type          = "t3.micro"
  subnet_id              = data.aws_subnets.default.ids[0]
  key_name               = "vmbkey"

  tags = {
    Name        = "Terraform-Student-VM"
    Environment = "Dev"
    Project     = "Experiment7"
  }

  volume_tags = {
    Name        = "Terraform-Student-VM"
    Environment = "Dev"
    Project     = "Experiment7"
  }
}
```

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "ap-southeast-2"
}

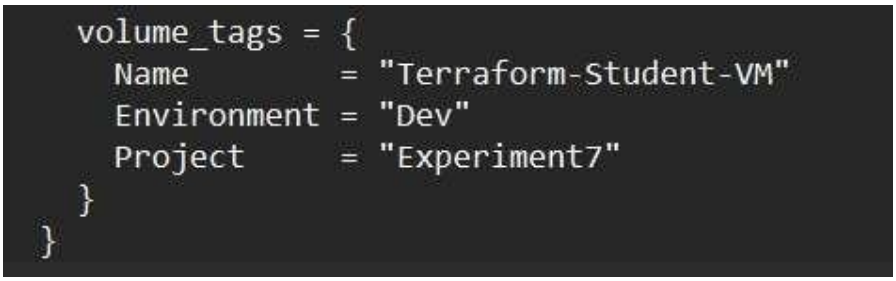
data "aws_ssm_parameter" "ubuntu_ami" {
  name = "/aws/service/canonical/ubuntu/server/22.04/stable/current/amd64/hvm/ebs-gp2/ami-id"
}

data "aws_vpc" "default" {
  default = true
}

data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

resource "aws_instance" "student_vm" {
  ami           = data.aws_ssm_parameter.ubuntu_ami.value
  instance_type = "t3.micro"
  subnet_id     = data.aws_subnets.default.ids[0]
  key_name      = "vmb-key"

  tags = {
    Name        = "Terraform-Student-VM"
    Environment = "Dev"
    Project     = "Experiment7"
  }
```



outputs.tf (Output State Extraction)

Terraform output "instance_id" { description

= "ID of the created EC2 instance" value

= aws_instance.student_vm.id

}

output "instance_type" { description = "Instance

type of the created VM" value =

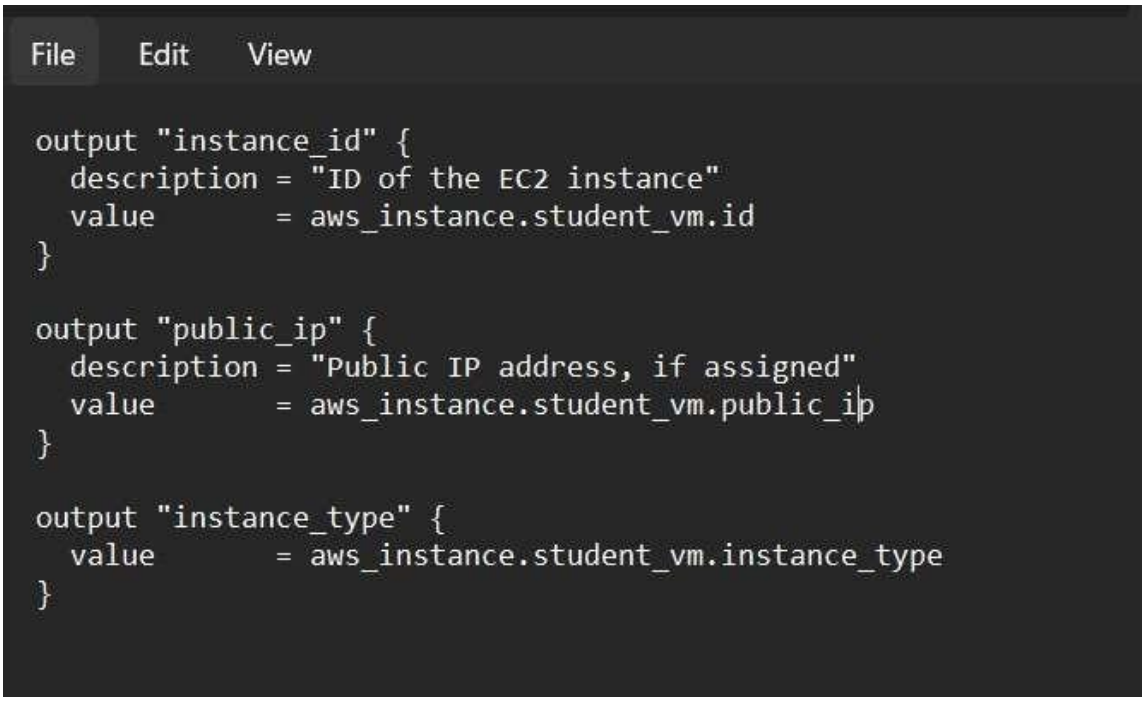
aws_instance.student_vm.instance_type

}

output "public_ip" { description = "Public IP address

assigned to the EC2 instance" value =

aws_instance.student_vm.public_ip }

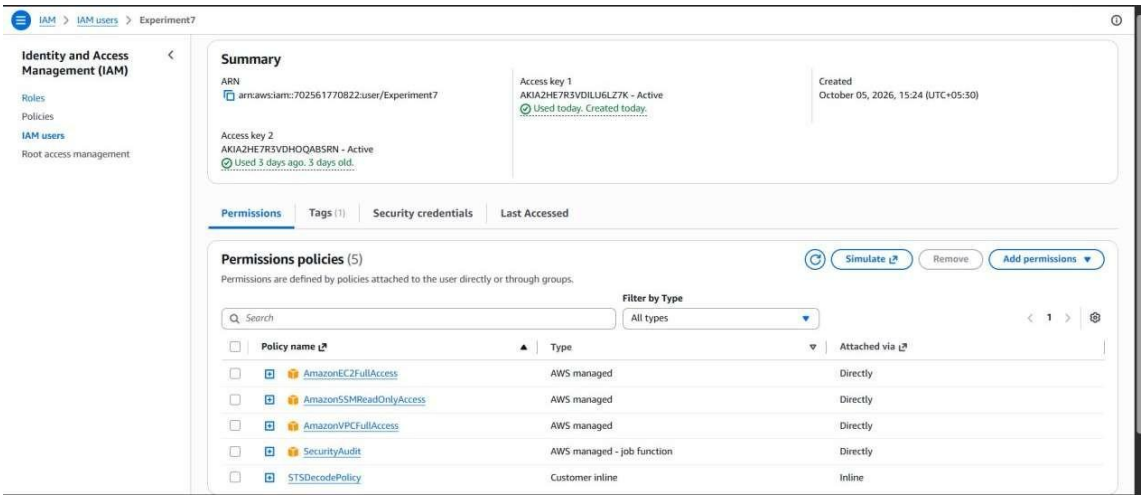


Step-by-Step Implementation Procedure

Phase 1: Local Environment & Credentials Configuration

Step 1: AWS IAM Credentials & Access Key Setup

1. Log in to the AWS Management Console and navigate to IAM > Users > Experiment7.
2. Select Security credentials, click Create access key, and choose Command Line Interface (CLI).
3. Generate and record the credentials:



Step 2: AWS CLI Authentication & Verification

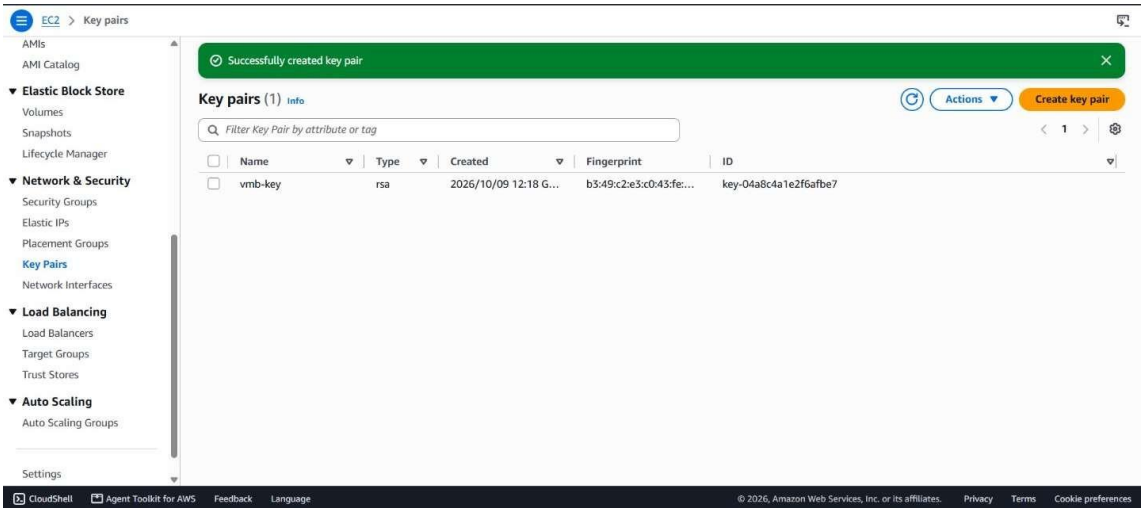
1. Open Windows PowerShell and run aws configure:
2. Verify identity via STS: PowerShell aws sts get-calleridentity

```
PS C:\Users\tanma> aws configure
AWS Access Key ID [****************BSRN]: AKIA2HE7R3VDILU6LZ7K
AWS Secret Access Key [****************wUZS]: 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
Default region name [ap-south-1]: ap-south-1
Default output format [json]: json
PS C:\Users\tanma> sts get-caller-identity
sts : The term 'sts' is not recognized as the name of a cmdlet, function, script file, or
operable program. Check the spelling of the name, or if a path was included, verify that
the path is correct and try again.
At line:1 char:1
+ sts get-caller-identity
+ ~~~
    + CategoryInfo          : ObjectNotFound: (sts:String) [], CommandNotFoundException
    + FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\tanma> aws sts get-caller-identity
{
    "UserId": "AIDA2HE7R3VDNRG6TUBQJ",
    "Account": "702561770822",
    "Arn": "arn:aws:iam::702561770822:user/Experiment7"
}
```

Step 3: EC2 Key Pair Generation in ap-southeast-2

1. Navigate to AWS Console > EC2 > Key Pairs in region apsoutheast-2.
2. Click Create key pair
3. Download and save vmb-key.pem.

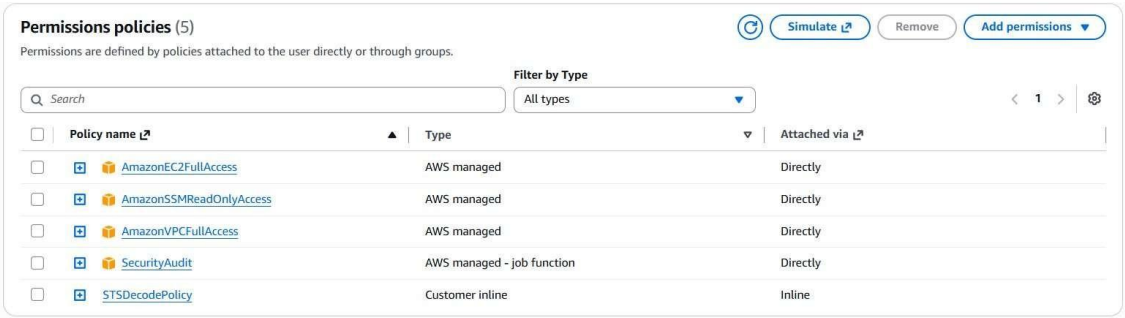


Phase 2: Resolving IAM Permissions & Policy Extensions

Step 4: Attaching SSM Read-Only Permissions

To enable Terraform to query the Canonical Ubuntu AMI ID dynamically from AWS Parameter Store, attach the SSM policy:

1. Go to IAM > Users > Experiment7 > Permissions.
2. Click Add permissions > Attach policies directly.
3. Search and select AmazonSSMReadOnlyAccess.
4. Click Add permissions.



Phase 3: Terraform Execution Lifecycle

Step 5: Directory Setup & Terraform Initialization

1. Create directory C:\Terraform and save main.tf and outputs.tf inside.
2. Open PowerShell, navigate to directory, and initialize backend:

PowerShell cd C:\Terraform terraform init

```
PS C:\Terraform> terraform init
Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.67.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Step 6: Code Validation & Execution Planning

1. Validate syntax:

PowerShell terraform

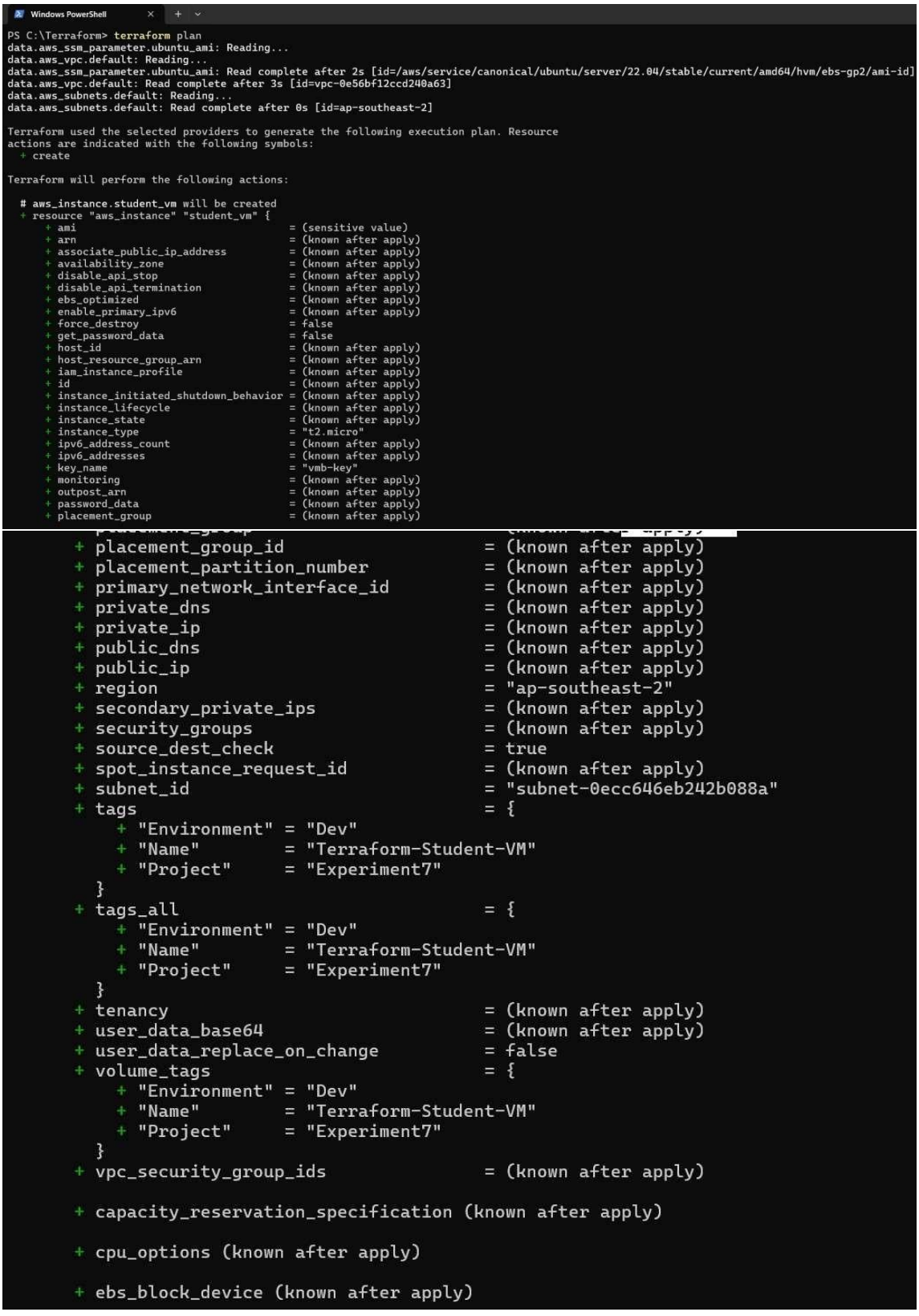
validate

2. Generate execution

plan: PowerShell terraform

plan

```
PS C:\Terraform> terraform validate
Success! The configuration is valid.
```



```
Windows PowerShell

PS C:\Terraform> terraform plan
data.aws_ssm_parameter.ubuntu_ami: Reading...
data.aws_vpc.default: Reading...
data.aws_ssm_parameter.ubuntu_ami: Read complete after 2s [id=/aws/service/canonical/ubuntu/server/22.04/stable/current/amd64/hvm/ebs-gp2/ami-id]
data.aws_vpc.default: Read complete after 3s [id=vpc-0e56bf12ccd240a63]
data.aws_subnets.default: Reading...
data.aws_subnets.default: Read complete after 0s [id=ap-southeast-2]

Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.student_vm will be created
  + resource "aws_instance" "student_vm" {
      + ami                                  = (sensitive value)
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t2.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "vmb-key"
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_group_id                   = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
      + private_ip                           = (known after apply)
      + public_dns                           = (known after apply)
      + public_ip                            = (known after apply)
      + region                               = "ap-southeast-2"
      + secondary_private_ips                = (known after apply)
      + security_groups                      = (known after apply)
      + source_dest_check                    = true
      + spot_instance_request_id             = (known after apply)
      + subnet_id                            = "subnet-0ecc646eb242b088a"
      + tags                                 = {
          + "Environment" = "Dev"
          + "Name"        = "Terraform-Student-VM"
          + "Project"     = "Experiment7"
        }
      + tags_all                             = {
          + "Environment" = "Dev"
          + "Name"        = "Terraform-Student-VM"
          + "Project"     = "Experiment7"
        }
      + tenancy                              = (known after apply)
      + user_data_base64                     = (known after apply)
      + user_data_replace_on_change          = false
      + volume_tags                          = {
          + "Environment" = "Dev"
          + "Name"        = "Terraform-Student-VM"
          + "Project"     = "Experiment7"
        }
      + vpc_security_group_ids               = (known after apply)

      + capacity_reservation_specification (known after apply)

      + cpu_options (known after apply)

      + ebs_block_device (known after apply)
```

```
      + enclave_options (known after apply)

      + ephemeral_block_device (known after apply)

      + instance_market_options (known after apply)

      + maintenance_options (known after apply)

      + metadata_options (known after apply)

      + network_interface (known after apply)

      + primary_network_interface (known after apply)

      + private_dns_name_options (known after apply)

      + root_block_device (known after apply)

      + secondary_network_interface (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + instance_id   = (known after apply)
  + public_ip     = (known after apply)

─────────────────────────────────────────────────────────────────────────────

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to
take exactly these actions if you run "terraform apply" now.
```

Step 7: Applying Infrastructure Provisioning

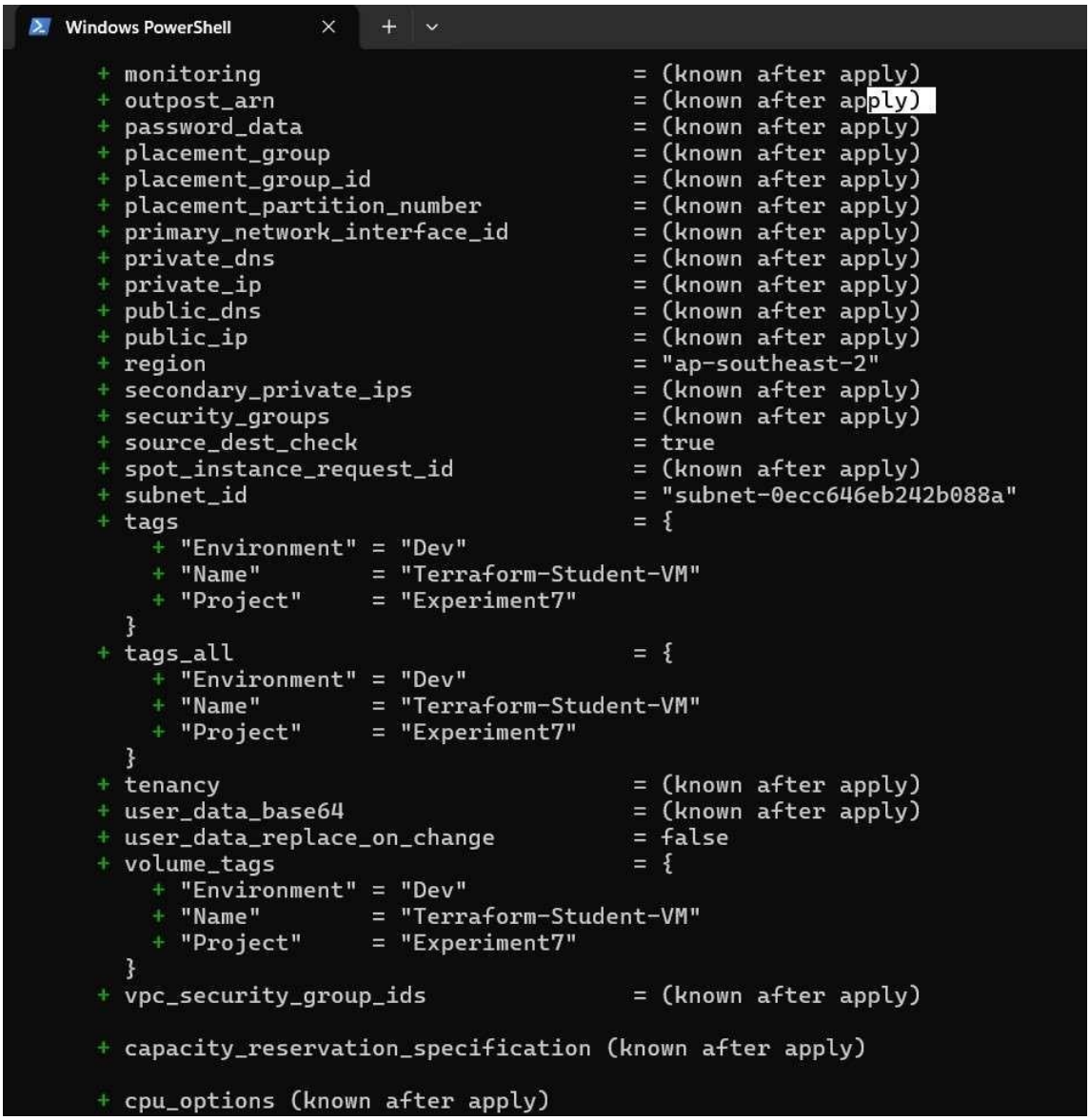
1. Execute apply command: PowerShell terraform apply
2. Enter yes at the approval prompt.
3. Confirm resource creation completion.

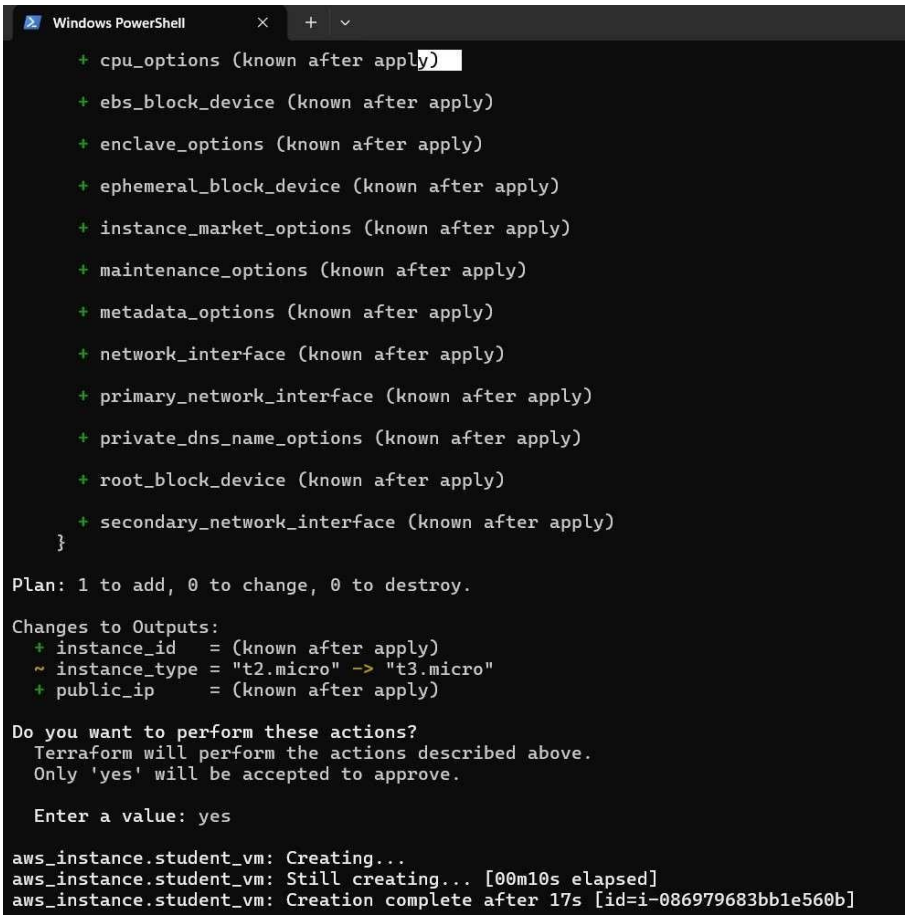
```
PS C:\Terraform> terraform apply
data.aws_vpc.default: Reading...
data.aws_ssm_parameter.ubuntu_ami: Reading...
data.aws_ssm_parameter.ubuntu_ami: Read complete after 1s [id=/aws/service/canonical/ubuntu/server/22.04/stable/current/amd64/hvm/ebs-gp2/ami-id]
data.aws_vpc.default: Read complete after 2s [id=vpc-0e56bf12ccd240a63]
data.aws_subnets.default: Reading...
data.aws_subnets.default: Read complete after 1s [id=ap-southeast-2]

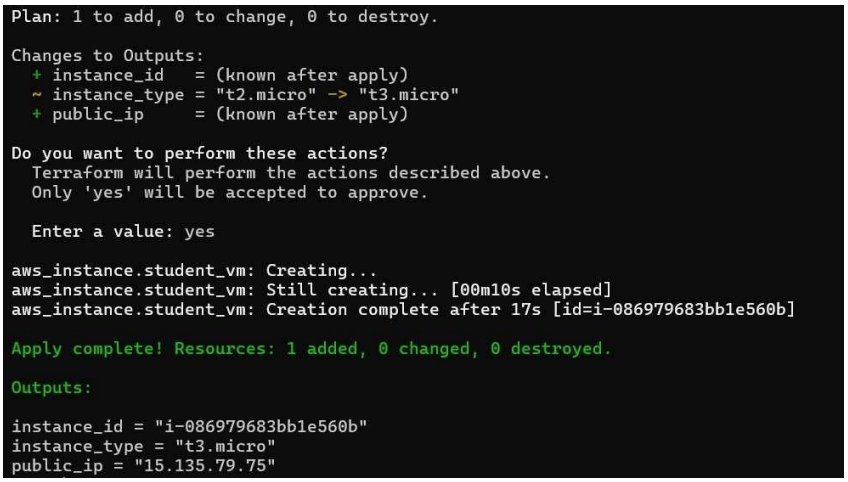
Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.student_vm will be created
  + resource "aws_instance" "student_vm" {
      + ami                                  = (sensitive value)
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t3.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "vmb-key"
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
```


Windows PowerShell
+ monitoring = (known after apply)
+ outpost_arn = (known after apply)
+ password_data = (known after apply)
+ placement_group = (known after apply)
+ placement_group_id = (known after apply)
+ placement_partition_number = (known after apply)
+ primary_network_interface_id = (known after apply)
+ private_dns = (known after apply)
+ private_ip = (known after apply)
+ public_dns = (known after apply)
+ public_ip = (known after apply)
+ region = "ap-southeast-2"
+ secondary_private_ips = (known after apply)
+ security_groups = (known after apply)
+ source_dest_check = true
+ spot_instance_request_id = (known after apply)
+ subnet_id = "subnet-0ecc646eb242b088a"
+ tags = {
+ "Environment" = "Dev"
+ "Name" = "Terraform-Student-VM"
+ "Project" = "Experiment7"
}
+ tags_all = {
+ "Environment" = "Dev"
+ "Name" = "Terraform-Student-VM"
+ "Project" = "Experiment7"
}
+ tenancy = (known after apply)
+ user_data_base64 = (known after apply)
+ user_data_replace_on_change = false
+ volume_tags = {
+ "Environment" = "Dev"
+ "Name" = "Terraform-Student-VM"
+ "Project" = "Experiment7"
}
+ vpc_security_group_ids = (known after apply)
+ capacity_reservation_specification (known after apply)
+ cpu_options (known after apply)





Phase 4: Verification & Teardown

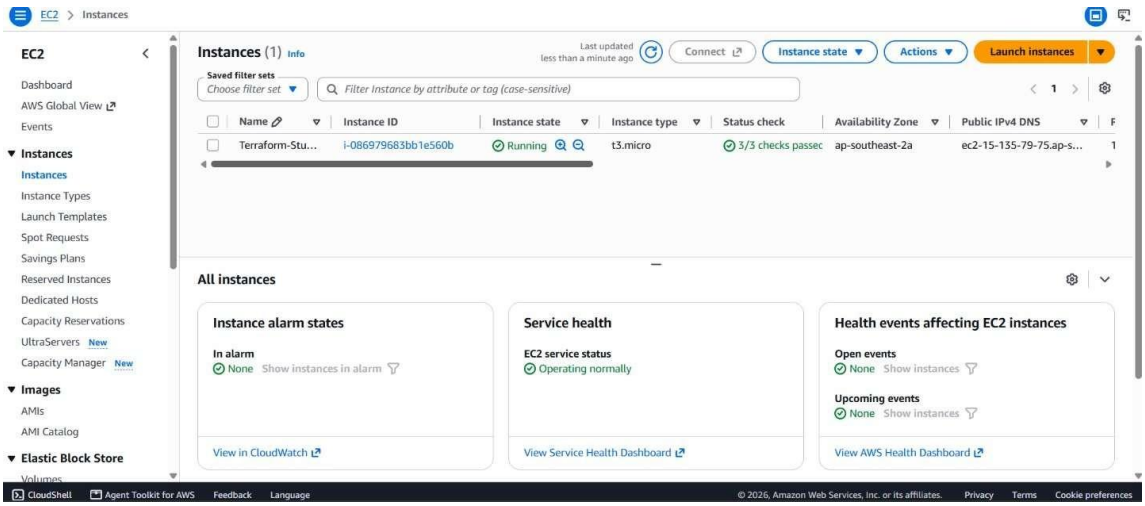
Step 8: Verifying Terraform Outputs & AWS EC2 Console State

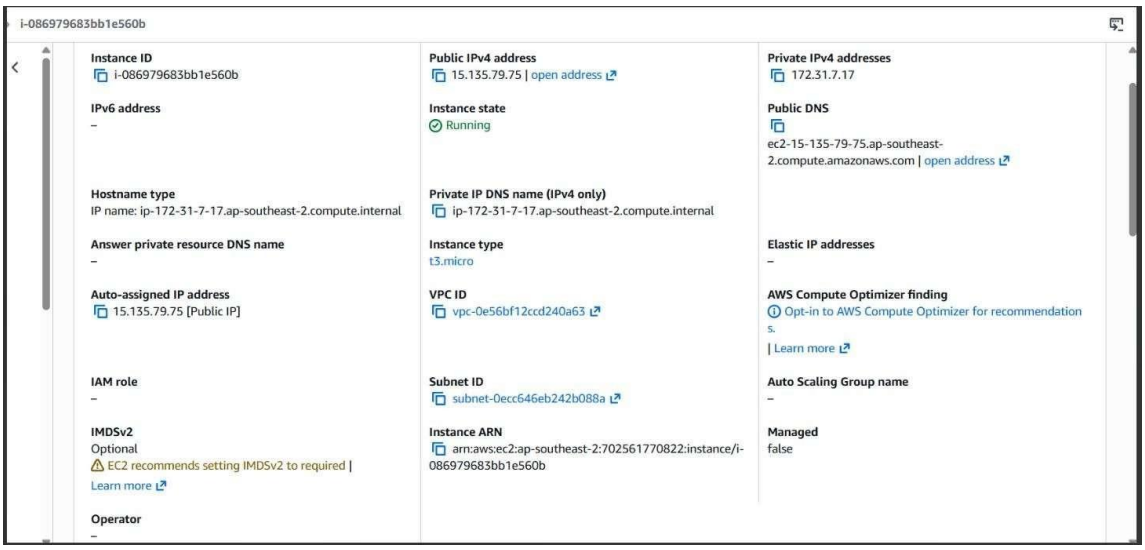
1. Run output command in PowerShell: PowerShell terraform output

2. Open browser and navigate to AWS EC2 Dashboard > Instances in region apsoutheast-2.

3. Verify that Terraform-Student-VM (i-086979683bb1e560b) is running with public IP 15.135.79.75.

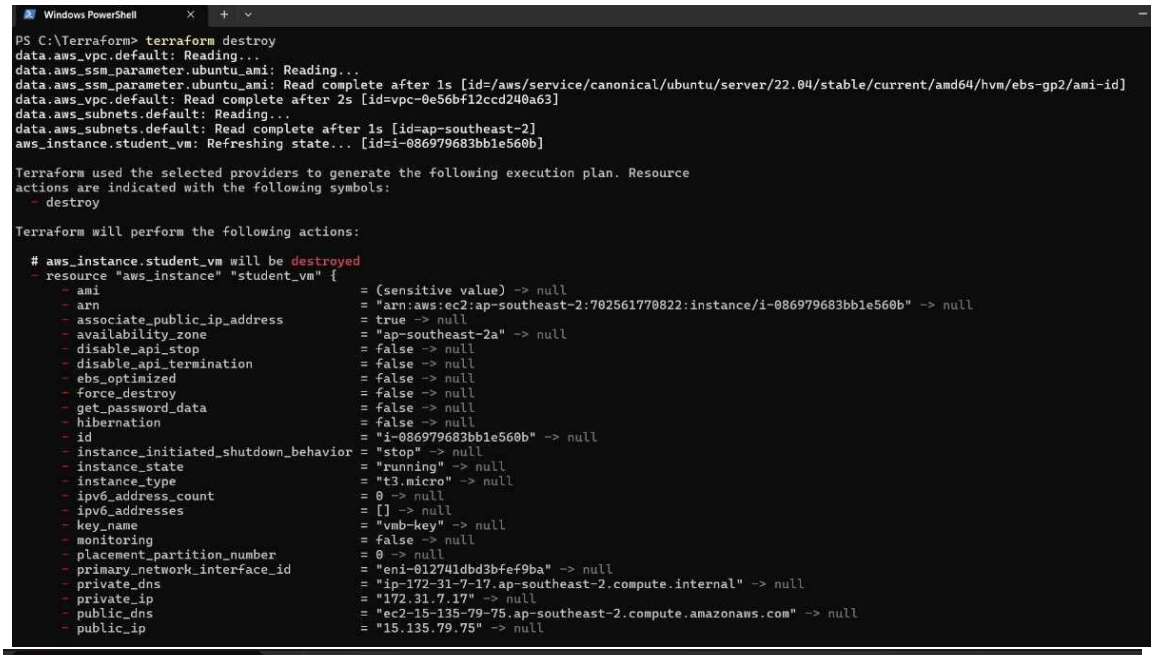
```
PS C:\Terraform> terraform output
instance_id = "i-086979683bb1e560b"
instance_type = "t3.micro"
public_ip = "15.135.79.75"
```

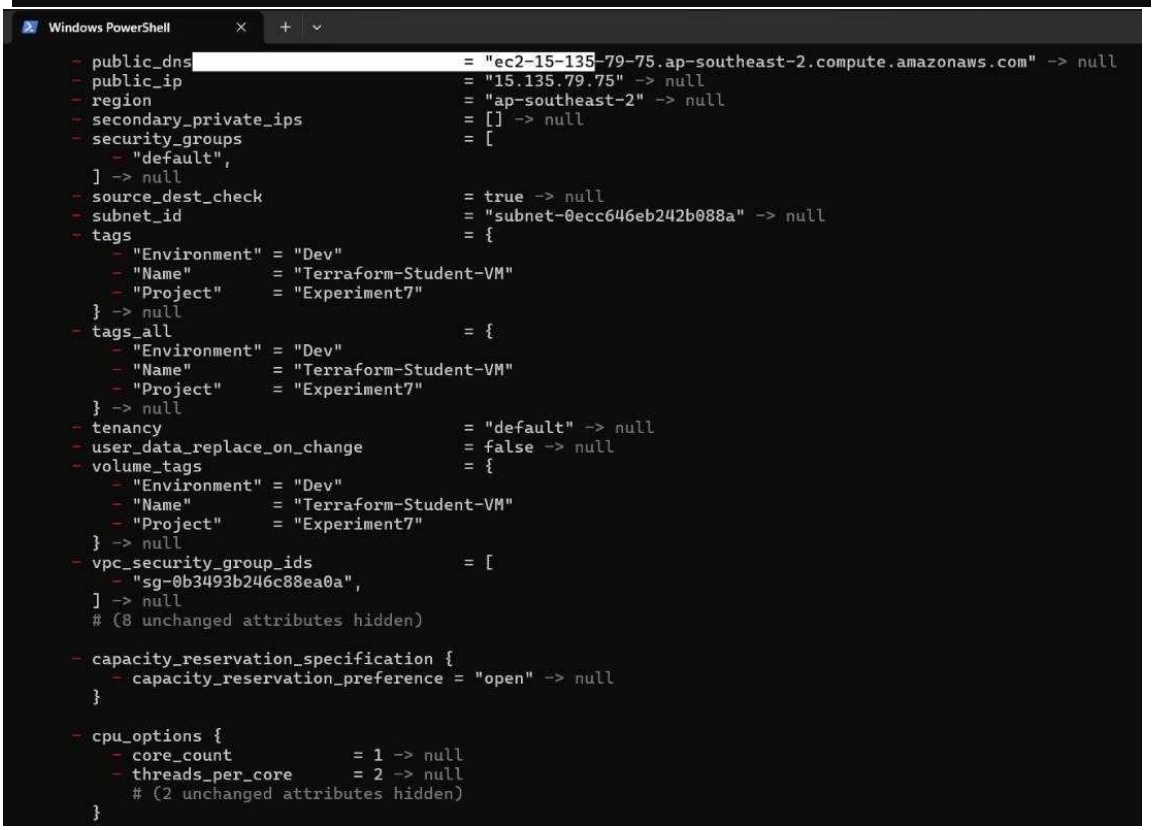




Step 9: Deprovisioning Cloud Resources (terraform destroy)

1. In PowerShell, initiate teardown: PowerShell terraform destroy
2. Type yes when prompted.
3. Confirm resource destruction.


Windows PowerShell
PS C:\Terraform> terraform destroy
data.aws_vpc.default: Reading...
data.aws_ssm_parameter.ubuntu_ami: Reading...
data.aws_ssm_parameter.ubuntu_ami: Read complete after 1s [id=/aws/service/canonical/ubuntu/server/22.04/stable/current/amd64/hvm/ebs-gp2/ami-id]
data.aws_vpc.default: Read complete after 2s [id=vpc-0e56bf12ccd240a63]
data.aws_subnets.default: Reading...
data.aws_subnets.default: Read complete after 1s [id=ap-southeast-2]
aws_instance.student_vm: Refreshing state... [id=i-086979683bb1e560b]
Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
- destroy
Terraform will perform the following actions:
aws_instance.student_vm will be destroyed
- resource "aws_instance" "student_vm" {
- ami = (sensitive value) -> null
- arn = "arn:aws:ec2:ap-southeast-2:702561770822:instance/i-086979683bb1e560b" -> null
- associate_public_ip_address = true -> null
- availability_zone = "ap-southeast-2a" -> null
- disable_api_stop = false -> null
- disable_api_termination = false -> null
- ebs_optimized = false -> null
- force_destroy = false -> null
- get_password_data = false -> null
- hibernation = false -> null
- id = "i-086979683bb1e560b" -> null
- instance_initiated_shutdown_behavior = "stop" -> null
- instance_state = "running" -> null
- instance_type = "t3.micro" -> null
- ipv6_address_count = 0 -> null
- ipv6_addresses = [] -> null
- key_name = "vmb-key" -> null
- monitoring = false -> null
- placement_partition_number = 0 -> null
- primary_network_interface_id = "eni-012741dbd3bfef9ba" -> null
- private_dns = "ip-172-31-7-17.ap-southeast-2.compute.internal" -> null
- private_ip = "172.31.7.17" -> null
- public_dns = "ec2-15-135-79-75.ap-southeast-2.compute.amazonaws.com" -> null
- public_ip = "15.135.79.75" -> null


Windows PowerShell
- public_dns = "ec2-15-135-79-75.ap-southeast-2.compute.amazonaws.com" -> null
- public_ip = "15.135.79.75" -> null
- region = "ap-southeast-2" -> null
- secondary_private_ips = [] -> null
- security_groups = [
"default",
] -> null
- source_dest_check = true -> null
- subnet_id = "subnet-0ecc646eb242b088a" -> null
- tags = {
- "Environment" = "Dev"
- "Name" = "Terraform-Student-VM"
- "Project" = "Experiment7"
} -> null
- tags_all = {
- "Environment" = "Dev"
- "Name" = "Terraform-Student-VM"
- "Project" = "Experiment7"
} -> null
- tenancy = "default" -> null
- user_data_replace_on_change = false -> null
- volume_tags = {
- "Environment" = "Dev"
- "Name" = "Terraform-Student-VM"
- "Project" = "Experiment7"
} -> null
- vpc_security_group_ids = [
- "sg-0b3493b246c88ea0a",
] -> null
(8 unchanged attributes hidden)
- capacity_reservation_specification {
- capacity_reservation_preference = "open" -> null
}
- cpu_options {
- core_count = 1 -> null
- threads_per_core = 2 -> null
(2 unchanged attributes hidden)
}


Windows PowerShell
}
- credit_specification {
- cpu_credits = "unlimited" -> null
}
- enclave_options {
- enabled = false -> null
}
- maintenance_options {
- auto_recovery = "default" -> null
}
- metadata_options {
- http_endpoint = "enabled" -> null
- http_protocol_ipv6 = "disabled" -> null
- http_put_response_hop_limit = 1 -> null
- http_tokens = "optional" -> null
- instance_metadata_tags = "disabled" -> null
}
- primary_network_interface {
- delete_on_termination = true -> null
- network_interface_id = "eni-012741dbd3bfef9ba" -> null
}
- private_dns_name_options {
- enable_resource_name_dns_a_record = false -> null
- enable_resource_name_dns_aaaa_record = false -> null
- hostname_type = "ip-name" -> null
}
- root_block_device {
- delete_on_termination = true -> null
- device_name = "/dev/sda1" -> null
- encrypted = false -> null
- iops = 100 -> null
- tags = {} -> null
- tags_all = {} -> null
- throughput = 0 -> null

```
Windows PowerShell
      - private_dns_name_options {
          - enable_resource_name_dns_a_record    = false -> null
          - enable_resource_name_dns_aaaa_record = false -> null
          - hostname_type                        = "ip-name" -> null
        }

      - root_block_device {
          - delete_on_termination = true -> null
          - device_name           = "/dev/sda1" -> null
          - encrypted             = false -> null
          - iops                  = 100 -> null
          - tags                  = {} -> null
          - tags_all              = {} -> null
          - throughput            = 0 -> null
          - volume_id             = "vol-0f27164e2f72bddb7" -> null
          - volume_size           = 8 -> null
          - volume_type           = "gp2" -> null
            # (1 unchanged attribute hidden)
        }
    }

Plan: 0 to add, 0 to change, 1 to destroy.

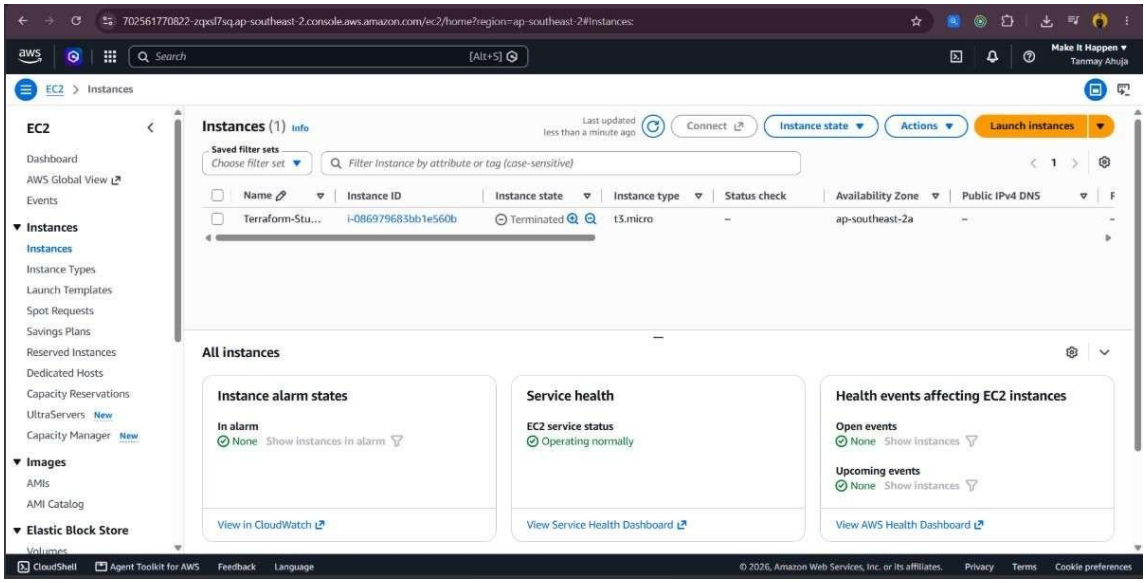
Changes to Outputs:
  - instance_id   = "i-086979683bb1e560b" -> null
  - instance_type = "t3.micro" -> null
  - public_ip     = "15.135.79.75" -> null

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

aws_instance.student_vm: Destroying... [id=i-086979683bb1e560b]
aws_instance.student_vm: Still destroying... [id=i-086979683bb1e560b, 00m10s elapsed]
aws_instance.student_vm: Still destroying... [id=i-086979683bb1e560b, 00m20s elapsed]
aws_instance.student_vm: Destruction complete after 21s

Destroy complete! Resources: 1 destroyed.
PS C:\Terraform>
```



Real-World Technical Challenges & Solutions

Sr No.	Error / Issue Encountered	Root Cause	Engineering Solution Implemented
1	403 UnauthorizedOperation on ec2:DescribeImages	Organization Service Control Policy (SCP: p-81tb9lnn) blocked AMI search queries.	Replaced data "aws_ami" block with dynamic Systems Manager (SSM) Parameter Store lookup (aws_ssm_parameter).
2	AccessDeniedException on ssm:GetParameter	User Experiment7 lacked permissions to read SSM paths.	Attached managed IAM policy AmazonSSMReadOnlyAccess to IAM user Experiment7.
3	403 Explicit Deny on ec2:RunInstances	SCP enforced mandatory tagging on both EC2 instances and root EBS volumes.	Configured both tags and volume_tags blocks (Environment="Dev", Project="Experiment7") in HCL code.
4	InvalidParameterCombination: Not eligible for Free Tier	Sandbox environment enforced t3.micro instance type over t2.micro.	Updated instance_type = "t3.micro" in main.tf.
5	InvalidAMIID.Malformed	Hardcoded AMI ID mismatch across AWS regions.	Bound AMI lookup dynamically to active region (ap-southeast-2) via SSM Parameter Store.

Conclusion

Through this experiment, hands-on mastery was acquired in defining, provisioning, managing, and deprovisioning AWS cloud infrastructure using HashiCorp Terraform as an Infrastructure as Code (IaC) tool. The experiment successfully satisfied Lab Outcome 4 (LO4) by executing automated resource management workflows on cloud infrastructure.

Applications of the Studied Technique in Industry

- Automated & Repeatable Deployments: Eliminates manual human error in cloud console management by declaring infrastructure state in version-controlled repositories.
- Multi-Cloud Architecture: Terraform provides a unified framework to provision resources across AWS, Azure, GCP, and Kubernetes clusters using declarative HCL scripts.
- Policy Compliance & Governance: Seamlessly integrates with enterprise Service Control Policies (SCPs) and security frameworks to enforce automated tagging, network isolation, and access controls.

Engineering Relevance

Infrastructure as Code forms the foundational backbone of modern DevOps paradigms, Site Reliability Engineering (SRE), and CI/CD pipelines. Programmatically controlling infrastructure lifecycle allows engineering teams to rapidly create ephemeral testing environments, scale compute capacity dynamically, maintain exact environment parity across staging and production, and implement disaster recovery protocols.